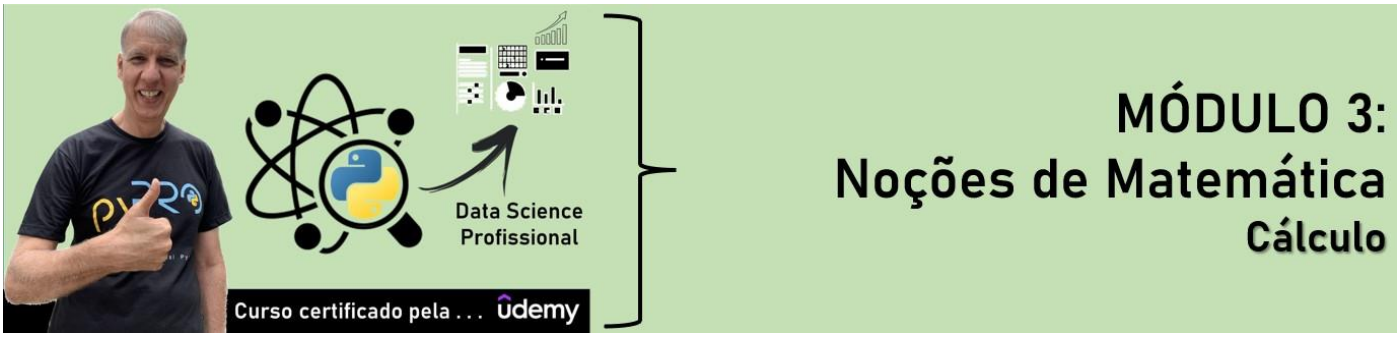


Noções básicas de Cálculo para Ciência de Dados

1. Introdução

2. Funções
2.1 – O que é uma função?
2.2 - Aplicação de uma função - Estudo do Vinho
2.3 - 2.3 Funções clássicas

3. Limites e Continuidade
3.1 Definição de limite
3.2 Regras de cálculo de limites
3.3 Continuidade de funções

4. Derivadas
4.1 Definição de derivada
4.2 Regras de derivação
4.3 Derivadas de funções elementares

5. Aplicações de Derivadas
5.1 Máximos e mínimos locais
5.2 Pontos de inflexão
5.3 Taxa de variação instantânea
5.4 Otimização de funções

1. Introdução

Seja bem-vindo(a) à esse módulo que abordará as Noções Básicas de Cálculo para Ciência de Dados!

Neste material, exploraremos os fundamentos matemáticos essenciais para a compreensão e aplicação da matemática ao mundo da Ciência de Dados.

O Cálculo é uma das disciplinas mais importantes e poderosas da matemática, e seu papel na Ciência de Dados é de fundamental importância. Ele oferece as ferramentas necessárias para analisar, modelar e prever fenômenos complexos, permitindo aos cientistas de dados obterem insights valiosos a partir de grandes conjuntos de dados.

Nosso foco inicial será nas três principais áreas do Cálculo: Funções, Limites e Derivadas. Esses conceitos formam a base sólida sobre a qual construiremos nosso conhecimento em análise de dados e modelagem matemática.

Funções: As funções são pilares centrais da matemática e da Ciência de Dados. Elas descrevem relações entre variáveis e nos permitem entender como uma grandeza depende de outra. Ao compreender funções, seremos capazes de descrever fenômenos e padrões em dados de maneira precisa e concisa.

Limites: O estudo de limites é fundamental para entender o comportamento das funções quando suas variáveis se aproximam de um determinado valor. Essa noção é crucial na análise de tendências, comportamentos de longo prazo e previsões de eventos em Ciência de Dados.

Derivadas: A derivada de uma função mede a taxa de variação de uma quantidade em relação a outra. Essa ferramenta poderosa nos permite encontrar pontos críticos, identificar padrões de crescimento e decréscimo, além de otimizar funções para atingir resultados desejados em aplicações de Ciência de Dados.

Compreender e aplicar esses conceitos de forma eficiente abrirá um mundo de possibilidades para você como cientista de dados. Através do Cálculo, seremos capazes de explorar algoritmos de aprendizado de máquina, entender processos dinâmicos em sistemas complexos e criar modelos matemáticos para tomar decisões inteligentes baseadas em dados.

Em nossa jornada, abordaremos conceitos teóricos fundamentais, exemplos práticos de aplicação em Ciência de Dados e exercícios para fortalecer sua compreensão. Acreditamos que a combinação do conhecimento em Cálculo com as habilidades em Ciência de Dados permitirá que você se destaque em um mundo cada vez mais orientado por dados.

Estamos animados para embarcar nessa jornada de aprendizado com você. Vamos começar a desvendar os mistérios do Cálculo aplicado à Ciência de Dados e abrir as portas para um futuro repleto de descobertas e inovações. Vamos em frente!

2. Funções

Uma função é uma relação matemática que associa cada elemento de um conjunto (domínio) a um único elemento de outro conjunto (contradomínio). Em outras palavras, uma função é uma regra que atribui valores de saída a valores de entrada.

2.1 O que é uma função?

Uma função é uma relação matemática que associa cada elemento de um conjunto (domínio) a um único elemento de outro conjunto (contradomínio). Em outras palavras, uma função é uma regra que atribui valores de saída a valores de entrada.

Uma função é representada por uma expressão matemática que envolve uma ou mais variáveis, e o resultado dessa expressão é o valor de saída da função. O conjunto de todos os valores possíveis de entrada é chamado de domínio, e o conjunto de todos os valores possíveis de saída é chamado de contradomínio.

Em termos visuais, podemos imaginar uma função como um "mecanismo" que transforma um valor de entrada em um valor de saída, seguindo uma determinada regra ou processo.

As funções são amplamente utilizadas em matemática, ciências, engenharia e em muitos outros campos, pois fornecem uma maneira de descrever e analisar relações entre variáveis e fazer previsões com base em dados.

2.2 Aplicação de uma função - Estudo do Vinho

Vamos utilizar o exemplo do vinho para ilustrar a aplicação de uma função em diversas óticas, como química, saúde e amizade. Vamos considerar que queremos analisar a relação entre a concentração de álcool no vinho e seu envelhecimento, e como esses fatores podem afetar diferentes aspectos.

Suponhamos que temos uma função matemática que descreve a concentração de álcool (em porcentagem) no vinho em relação ao tempo de envelhecimento (em anos):

```
Concentracao_de_alcool(envelhecimento) = 14 - 0.2 * envelhecimento
```

Nessa função, envelhecimento é a variável de entrada (domínio) que representa o tempo em anos, e Concentracao_de_alcool(envelhecimento) é o valor de saída (contradomínio) que representa a concentração de álcool em porcentagem.

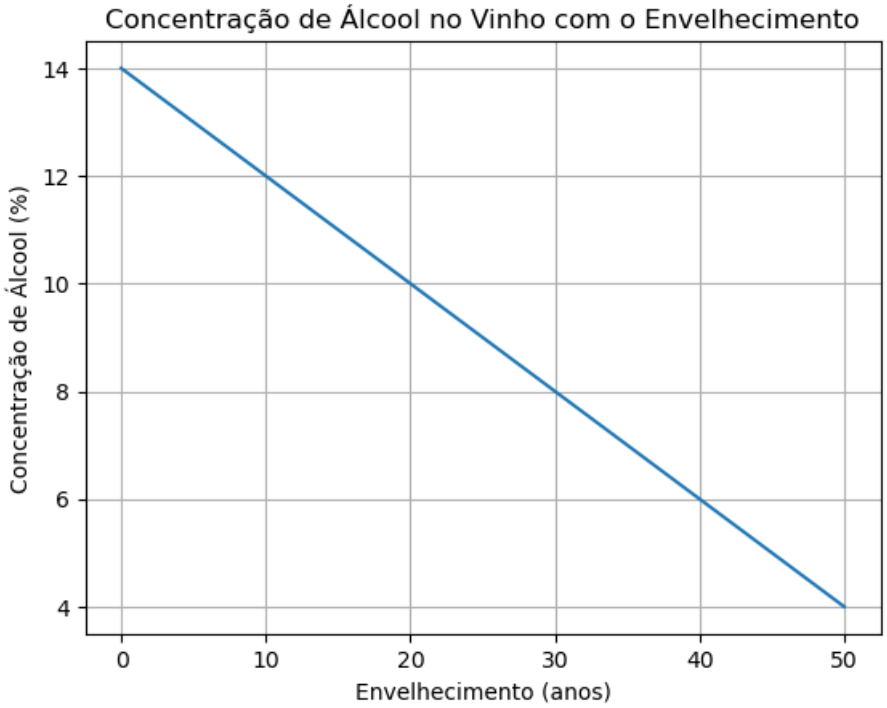
Agora, vamos explorar algumas análises utilizando essa função em Python e o Jupyter Notebook.

Exemplo de Análises com Python e Jupyter Notebook:

Vamos utilizar a biblioteca matplotlib para visualizar os gráficos das análises do vinho.

```
import matplotlib.pyplot as plt
import numpy as np

# Função que descreve a concentração de álcool no vinho em relação ao envelhecimento
def Concentracao_de_alcool(envelhecimento):
    return 14 - 0.2 * envelhecimento

# Valores para o envelhecimento do vinho (de 0 a 50 anos)
envelhecimento = np.linspace(0, 50, 100)

# Calculando os valores de concentração de álcool usando a função
concentracao = Concentracao_de_alcool(envelhecimento)
```

```
# Plotando o gráfico da concentração de álcool versus envelhecimento
plt.plot(envelhecimento, concentracao)
plt.xlabel('Envelhecimento (anos)')
plt.ylabel('Concentração de Álcool (%)')
plt.title('Concentração de Álcool no Vinho com o Envelhecimento')
plt.grid(True)
plt.show()
```



Esse gráfico mostra como a concentração de álcool no vinho diminui ao longo do tempo de envelhecimento.

Agora, vamos explorar outras análises do vinho sob diferentes óticas:

Análise Farmacológica:

Digamos que temos outra função que descreve como a concentração de álcool no sangue de uma pessoa diminui ao longo do tempo após a ingestão de uma quantidade específica de vinho:

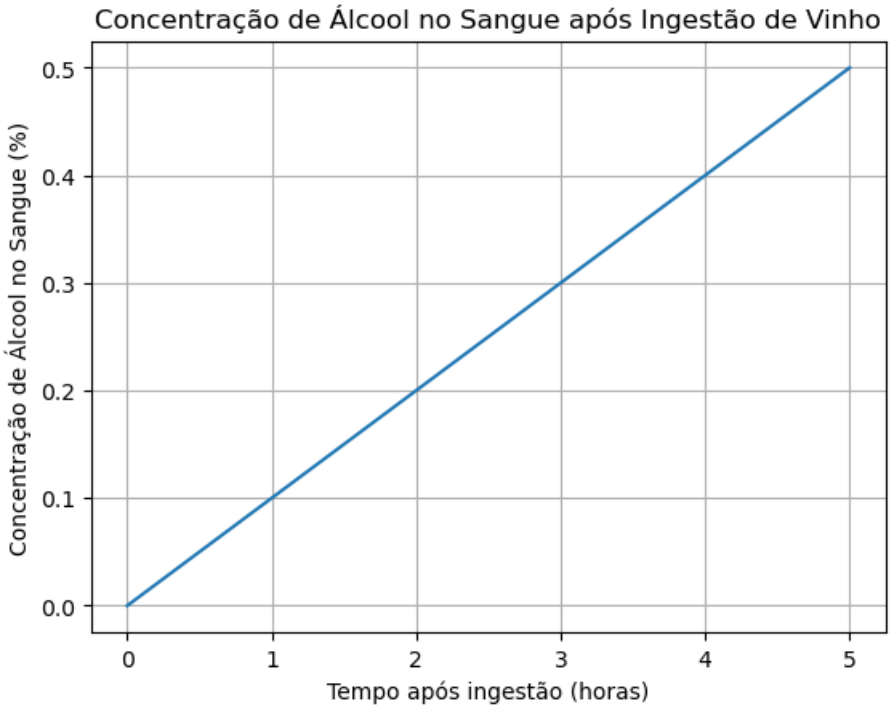
```
Concentracao_no_sangue(tempo) = 0.1 * tempo
```

Nessa função, tempo é a variável de entrada (domínio) que representa o tempo em horas após a ingestão, e Concentracao_no_sangue(tempo) é o valor de saída (contradomínio) que representa a concentração de álcool no sangue.

Vamos plotar o gráfico dessa função em Python:

```
# Função que descreve a concentração de álcool no sangue em relação ao tempo após
ingestão
def Concentracao_no_sangue(tempo):
    return 0.1 * tempo

# Valores para o tempo após ingestão do vinho (de 0 a 5 horas)
tempo = np.linspace(0, 5, 100)

# Calculando os valores de concentração de álcool no sangue usando a função
```

```
concentracao_sangue = Concentracao_no_sangue(tempo)

# Plotando o gráfico da concentração de álcool no sangue versus tempo após ingestão
plt.plot(tempo, concentracao_sangue)
plt.xlabel('Tempo após ingestão (horas)')
plt.ylabel('Concentração de Álcool no Sangue (%)')
plt.title('Concentração de Álcool no Sangue após Ingestão de Vinho')
plt.grid(True)
plt.show()
```



Análise Social - Amizade e Vinho:

Vamos considerar uma função que descreve o nível de amizade (de 0 a 100) entre duas pessoas em relação à quantidade de vinho compartilhado em um período de tempo:

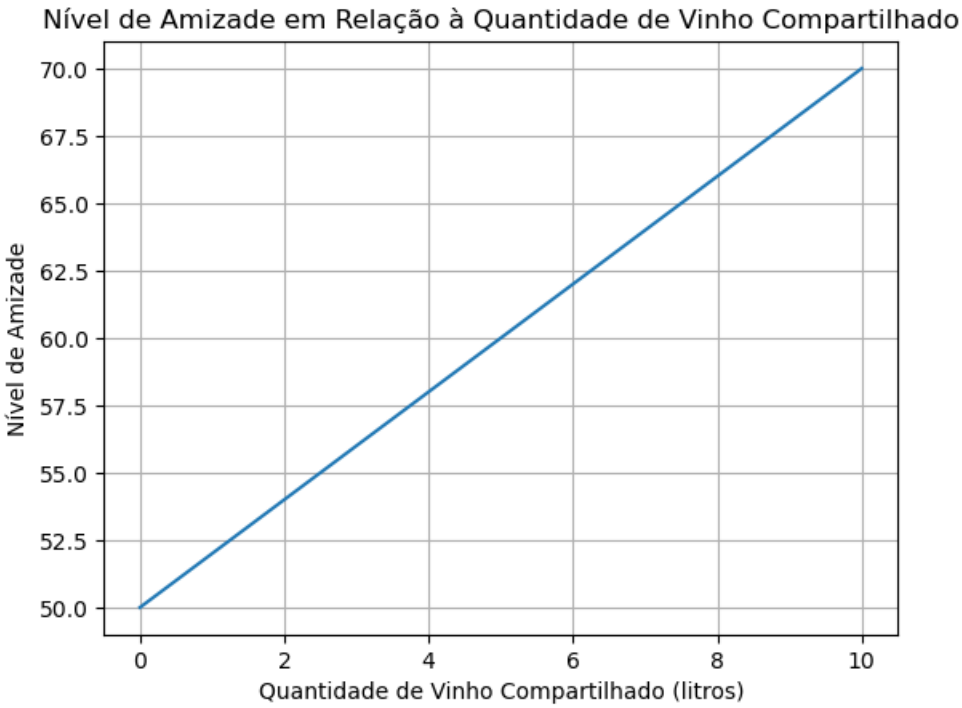
```
Nivel_de_amizade(qtd_vinho_compartilhado) = 50 + 2 * qtd_vinho_compartilhado
```

Nessa função, qtd_vinho_compartilhado é a variável de entrada (domínio) que representa a quantidade de vinho compartilhado em litros, e Nivel_de_amizade(qtd_vinho_compartilhado) é o valor de saída (contradomínio) que representa o nível de amizade entre as pessoas.

Vamos plotar o gráfico dessa função em Python:

```
# Função que descreve o nível de amizade em relação à quantidade de vinho
compartilhado
def Nivel_de_amizade(qtd_vinho_compartilhado):
    return 50 + 2 * qtd_vinho_compartilhado

# Valores para a quantidade de vinho compartilhado (de 0 a 10 litros)
qtd_vinho_compartilhado = np.linspace(0, 10, 100)

# Calculando os valores de nível de amizade usando a função
nivel_amizade = Nivel_de_amizade(qtd_vinho_compartilhado)

# Plotando o gráfico do nível de amizade versus quantidade de vinho compartilhado
plt.plot(qtd_vinho_compartilhado, nivel_amizade)
```

```
plt.xlabel('Quantidade de Vinho Compartilhado (litros)')
plt.ylabel('Nível de Amizade')
plt.title('Nível de Amizade em Relação à Quantidade de Vinho Compartilhado')
plt.grid(True)
plt.show()
```



Esses exemplos ilustram como as funções podem ser aplicadas para analisar e entender o vinho sob diversas óticas, como química, saúde e amizade. Em cada análise, temos uma função diferente que descreve a relação entre as variáveis de interesse. Os gráficos ajudam a visualizar essas relações e proporcionam uma melhor compreensão dos dados.

2.3 Funções clássicas

Existem várias funções matemáticas clássicas que são frequentemente encontradas em problemas e aplicações. Algumas das funções clássicas incluem:

- **Função Linear**: f(x) = mx + b, onde m é a inclinação (coeficiente angular) e b é o intercepto no eixo y.
- **Função Quadrática**: f(x) = ax^2 + bx + c, onde a, b e c são constantes e a ≠ 0.
- **Função Exponencial**: f(x) = a^x, onde a é uma constante positiva.
- **Função Logarítmica**: f(x) = log_a(x), onde a é a base do logaritmo e x > 0.

Definição e exemplos de Funções em Python

Em Python, você pode definir funções utilizando a sintaxe def. Uma função em Python recebe um conjunto de argumentos (valores de entrada) e pode retornar um valor (valor de saída). Utilizaremos a biblioteca matplotlib para plotar os gráficos. Antes de prosseguir, certifique-se de ter a biblioteca instalada em seu ambiente. Caso não tenha, você pode instalá-la usando o comando `!pip install matplotlib.`

a) Exemplo de Função Linear

Vamos criar uma função linear em Python que recebe o valor de x, a inclinação m e o intercepto b, e retorna o valor de y de acordo com a fórmula f(x) = mx + b.

```
def funcao_linear(x, m, b):
    y = m * x + b
    return y
```

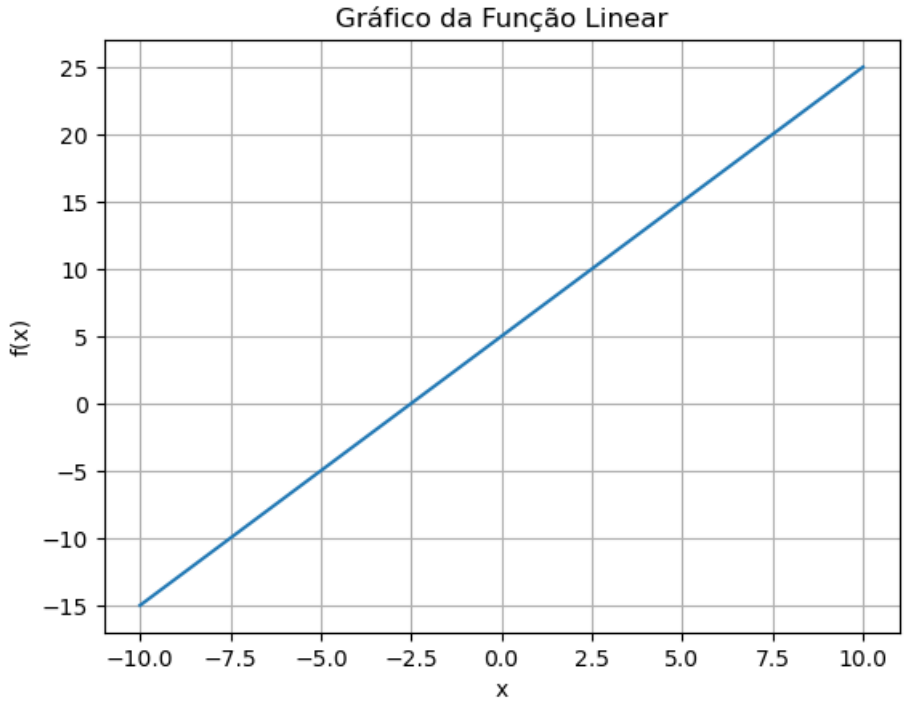
Agora, podemos usar essa função para calcular valores de saída y com base nos valores de entrada x, m e b.

```
# Exemplo de uso da função linear
x = 2
m = 3
b = 1

resultado = funcao_linear(x, m, b)
print(f"Para x = {x}, f(x) = {resultado}")
```

```
Para x = 2, f(x) = 7
```

Plotando a Função Linear:

```
import matplotlib.pyplot as plt

def funcao_linear(x, m, b):
    return m * x + b

# Valores para a função linear
m = 2
b = 5
x = range(-10, 11)  # Valores de x de -10 a 10

# Calculando os valores de y
y = [funcao_linear(i, m, b) for i in x]

# Plotando o gráfico
plt.plot(x, y)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Gráfico da Função Linear')
plt.grid(True)
plt.show()
```



b) Exemplo de Função Quadrática

Vamos criar uma função em Python que recebe o valor de x, os coeficientes a, b e c, e retorna o valor de y de acordo com a fórmula f(x) = ax^2 + bx + c.

```
def funcao_quadratica(x, a, b, c):
    y = a * x**2 + b * x + c
    return y
```

Agora, podemos usar essa função para calcular valores de saída y com base nos valores de entrada x, a, b e c.

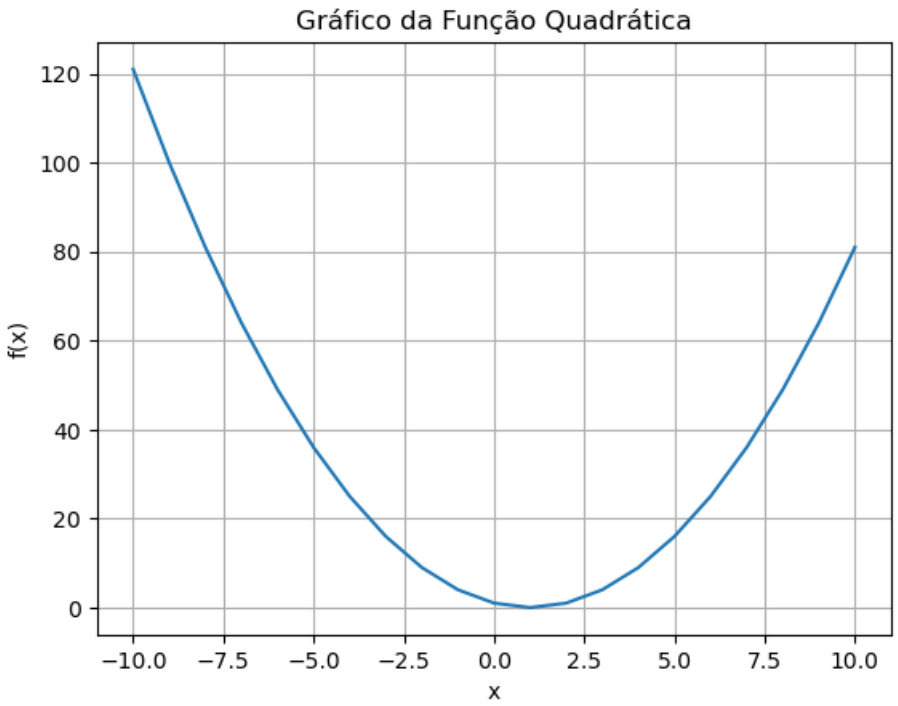
```
# Exemplo de uso da função quadrática
x = 3
a = 2
b = -3
c = 1

resultado = funcao_quadratica(x, a, b, c)
print(f"Para x = {x}, f(x) = {resultado}")
```

```
Para x = 3, f(x) = 10
```

Plotando a Função Quadrática:

```
import matplotlib.pyplot as plt

def funcao_quadratica(x, a, b, c):
    return a * x**2 + b * x + c

# Valores para a função quadrática
a = 1
```

```
b = -2
c = 1
x = range(-10, 11)  # Valores de x de -10 a 10

# Calculando os valores de y
y = [funcao_quadratica(i, a, b, c) for i in x]

# Plotando o gráfico
plt.plot(x, y)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Gráfico da Função Quadrática')
plt.grid(True)
plt.show()
```



c) Exemplo de Função Exponencial

Em Python, a função exponencial pode ser facilmente implementada usando o operador **.

```
def funcao_exponencial(x, a):
    y = a ** x
    return y
```

Agora, podemos usar essa função para calcular valores de saída y com base nos valores de entrada x e a.

```
# Exemplo de uso da função exponencial
x = 2
a = 3

resultado = funcao_exponencial(x, a)
print(f"Para x = {x}, f(x) = {resultado}")
```

```
Para x = 2, f(x) = 9
```

Plotando a Função Exponencial:

```
import matplotlib.pyplot as plt

def funcao_exponencial(x, a):
    return a ** x

# Valores para a função exponencial
a = 2
x = range(-10, 11)  # Valores de x de -10 a 10

# Calculando os valores de y
y = [funcao_exponencial(i, a) for i in x]

# Plotando o gráfico
plt.plot(x, y)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Gráfico da Função Exponencial')
plt.grid(True)
plt.show()
```

d) Exemplo de Função Logarítmica

Em Python, a função logarítmica pode ser implementada utilizando a biblioteca math que já possui a função log().

```
import math

def funcao_logaritmica(x, a):
    y = math.log(x, a)
    return y
```

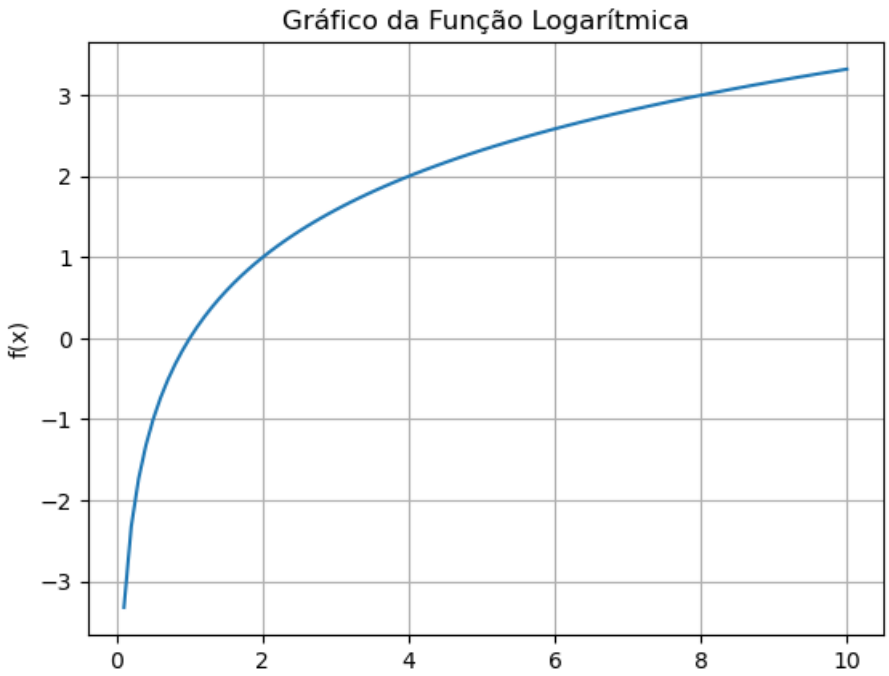
Agora, podemos usar essa função para calcular valores de saída y com base nos valores de entrada x e a.

```
# Exemplo de uso da função logarítmica
x = 10
a = 2

resultado = funcao_logaritmica(x, a)
print(f"Para x = {x}, f(x) = {resultado}")
```

```
Para x = 10, f(x) = 3.3219280948873626
```

Plotando a Função Logarítmica:

```
import matplotlib.pyplot as plt
import numpy as np

def funcao_logaritmica(x, a):
    return np.log(x) / np.log(a)

# Valores para a função logarítmica
a = 2
x = np.linspace(0.1, 10, 100)  # Valores de x de 0.1 a 10 (o log não é definido para
x=0)

# Calculando os valores de y
y = [funcao_logaritmica(i, a) for i in x]

# Plotando o gráfico
plt.plot(x, y)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Gráfico da Função Logarítmica')
plt.grid(True)
plt.show()
```



Com esses exemplos, você pode ver como é simples definir funções em Python e usá-las para calcular valores de saída com base nos valores de entrada.

3. Limites e Continuidade

Os limites são fundamentais para entender o comportamento das funções quando se aproximam de um determinado valor ou tendem ao infinito. A continuidade de uma função é outra propriedade importante para garantir a existência de limites.

3.1. Definição de Limite
O conceito de limite é fundamental para entender o comportamento das funções quando se aproximam de um determinado valor ou tendem ao infinito. Na Ciência de Dados, os limites são usados em várias aplicações, como cálculo de taxas de variação e análise de tendências.

Formalmente temos...
Um limite é definido como o valor que uma função se aproxima quando sua variável independente se aproxima de um determinado valor. Matematicamente, podemos representar o limite da função f(x) quando x se aproxima de a como:

```
lim x->a f(x) = L
```

Isso significa que queremos saber o valor para o qual a função tende quando x se aproxima de a. O limite pode nos ajudar a entender o comportamento de uma função em pontos específicos, identificar assíntotas verticais e horizontais e calcular valores de inclinação instantânea (derivadas).

Notação de Limite
Existem três notações comumente usadas para representar limites:

Notação de limite padrão:

```
lim x->a f(x) = L
```

Notação de limite com "h":

```
lim h->0 f(a+h) = L
```

Notação de limite com "x" tendendo ao infinito:

```
lim x->∞ f(x) = L
```

Exemplo de Codificação em Jupyter Notebook

Aqui está um exemplo de como calcular um limite em Python usando a biblioteca SymPy em um Jupyter Notebook:

```
from sympy import Symbol, limit, sin

# Definindo uma variável simbólica
x = Symbol('x')

# Calculando um limite
limite = limit(sin(x) / x, x, 0)

print("Limite:", limite)
```

```
Limite: 1
```

Neste exemplo, estamos calculando o limite da função sin(x)/x quando x se aproxima de 0. A biblioteca SymPy é usada para realizar esse cálculo.

3.2. O Cálculo de Limites

Existem várias técnicas para calcular limites, dependendo do tipo de função e da expressão envolvida. A seguir apresentaremos algumas das técnicas mais comuns:

- **Simplificação Direta**: Substituir o valor de x no limite e simplificar a expressão. Isso é especialmente útil quando temos limites envolvendo funções polinomiais ou funções racionais.

- **Fatoração**: Fatorar a expressão e simplificar. Isso pode ser útil quando temos limites indeterminados, como 0/0 ou ∞/∞.

- **Uso de limites trigonométricos conhecidos**: Existem limites trigonométricos que são amplamente conhecidos e podem ser usados para simplificar as expressões.

- **Uso de propriedades dos limites**: Existem regras e propriedades que podem ser aplicadas para simplificar limites, como a regra do limite da soma, diferença, produto e quociente.

A seguir, vamos exemplificar o cálculo de limites usando algumas das técnicas mencionadas.

Vamos considerar o seguinte exemplo para calcular o limite da função f(x) à medida que x se aproxima de um valor c:

```
f(x) = (x^2 - 4) / (x - 2)
```

Vamos calcular o limite `lim_(x → 2) f(x)` usando a técnica de simplificação direta:

```
import sympy as sp

# Definindo a variável x
x = sp.symbols('x')

# Definindo a função f(x)
f = (x**2 - 4) / (x - 2)

# Calculando o limite lim_(x → 2) f(x)
limite = sp.limit(f, x, 2)
print("Limite:", limite)
```

A saída será:

```
Limite: 4
```

Neste exemplo, ao calcular o limite lim_(x → 2) f(x), obtemos o valor 4. Isso significa que a função tende a 4 quando x se aproxima de 2.

Este é apenas um exemplo simples de como calcular limites usando Python e a biblioteca sympy, que nos permite trabalhar simbolicamente com expressões matemáticas.

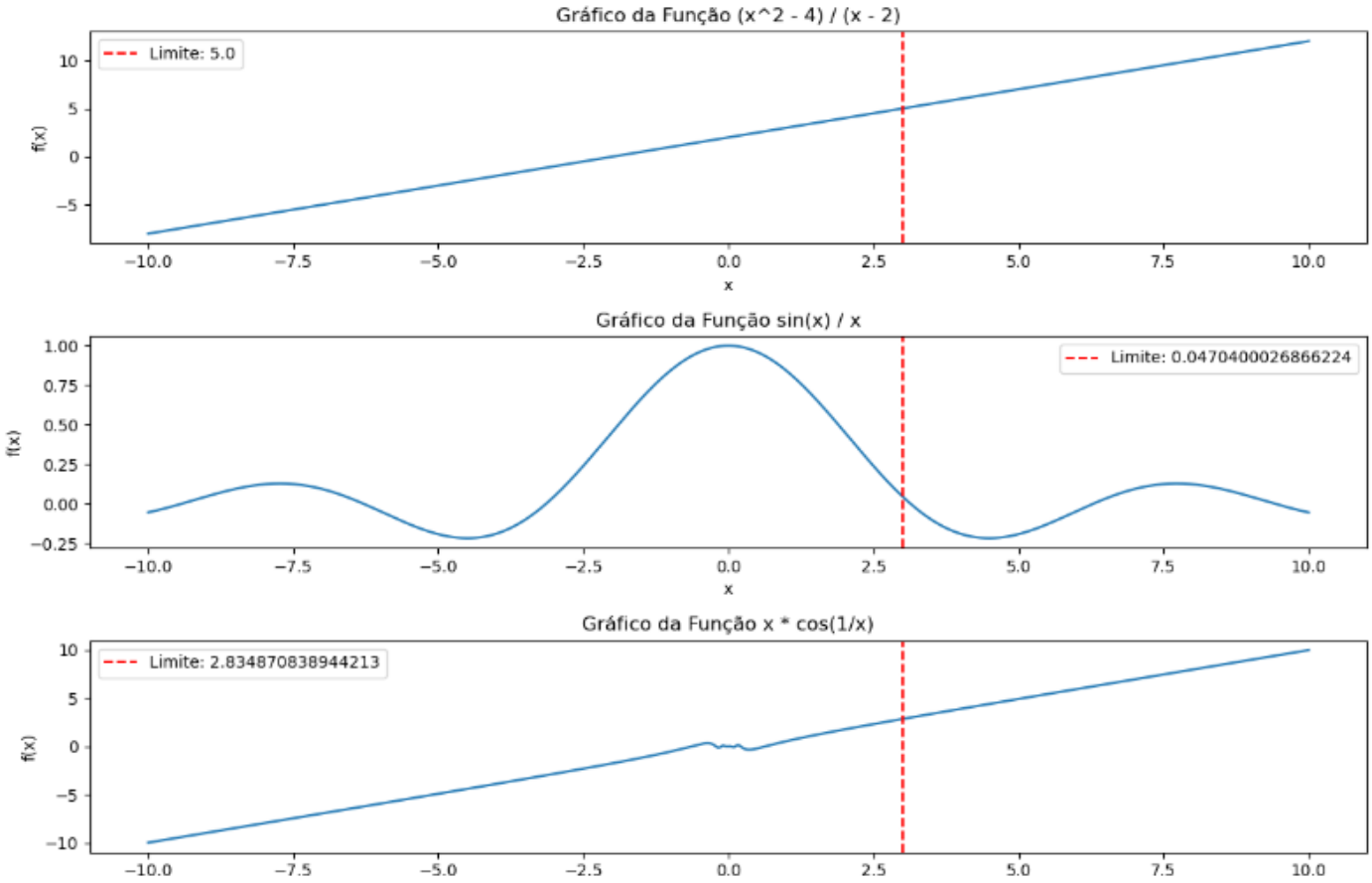
Gráficos para as Definições de Limite:

Vamos ilustrar graficamente o comportamento de algumas funções quando o valor de x se aproxima de um determinado valor c. Para isso, vamos usar as técnicas de simplificação direta e fatoração para calcular os limites.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
# Funções que queremos analisar
def f1(x):
    return (x**2 - 4) / (x - 2)

def f2(x):
    return np.sin(x) / x

def f3(x):
    return x * np.cos(1/x)

# Valores para x
x = np.linspace(-10, 10, 500)

# Valor de c para os limites
c = 3

# Cálculo dos limites usando as funções definidas
limite_f1 = f1(c)
limite_f2 = f2(c)
limite_f3 = f3(c)

# Plotando os gráficos das funções
plt.figure(figsize=(12, 8))

plt.subplot(3, 1, 1)
plt.plot(x, f1(x))
plt.axvline(x=c, color='red', linestyle='--', label=f'Limite: {limite_f1}')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Gráfico da Função (x^2 - 4) / (x - 2)')
plt.legend()

plt.subplot(3, 1, 2)
plt.plot(x, f2(x))
plt.axvline(x=c, color='red', linestyle='--', label=f'Limite: {limite_f2}')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Gráfico da Função sin(x) / x')
plt.legend()

plt.subplot(3, 1, 3)
plt.plot(x, f3(x))
plt.axvline(x=c, color='red', linestyle='--', label=f'Limite: {limite_f3}')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Gráfico da Função x * cos(1/x)')
plt.legend()

plt.tight_layout()
plt.show()
```

No código acima, definimos três funções (f1, f2 e f3) que correspondem a diferentes expressões matemáticas. Em seguida, calculamos os valores dos limites quando x se aproxima de c = 3 utilizando as técnicas de simplificação direta e fatoração.



Em cada gráfico, a linha vermelha pontilhada representa o valor de c e a legenda mostra o valor do limite correspondente.

Mais um exemplo de cálculo de Limite...

Aqui está um exemplo de como aplicar as regras de cálculo de limites em Python usando a biblioteca SymPy em um Jupyter Notebook:

```
from sympy import Symbol, limit, sin, cos

# Definindo uma variável simbólica
x = Symbol('x')

# Aplicando regras de cálculo de limites
f = sin(x) * cos(x)
limite = limit(f, x, 0)

print("Limite:", limite)
```

```
Limite: 0
```

Neste exemplo, estamos calculando o limite da função sin(x) * cos(x) quando x se aproxima de 0. A biblioteca SymPy é usada para simplificar a expressão e aplicar as regras de cálculo de limites.

3.3. Continuidade de Funções

A continuidade de uma função é uma propriedade importante que descreve a existência de limites e a suavidade da função em um determinado intervalo. Na Ciência de Dados, a continuidade é relevante para garantir a validade dos modelos e das análises realizadas.

Definição de Função Contínua
Uma função é considerada contínua em um ponto se o limite da função existe nesse ponto e é igual ao valor da função naquele ponto. Uma função é considerada contínua em um intervalo se for contínua em cada ponto desse intervalo.

Tipos de Descontinuidades
Existem diferentes tipos de descontinuidades que podem ocorrer em funções:

- Descontinuidade Removível: ocorre quando uma função possui uma descontinuidade em um ponto, mas pode ser tornada contínua atribuindo um valor adequado à função nesse ponto.
- Descontinuidade de Salto: ocorre quando uma função tem uma diferença finita entre os limites do ponto de descontinuidade.
- Descontinuidade Infinita: ocorre quando uma função tem um limite infinito em um ponto de descontinuidade.

Exemplo de Codificação em Jupyter Notebook

Aqui está um exemplo de como verificar a continuidade de uma função em Python usando a biblioteca SymPy em um Jupyter Notebook:

```
from sympy import Symbol, sin, pi, oo

# Definindo uma variável simbólica
x = Symbol('x')

# Verificando a continuidade da função
f = sin(x)
continuidade_em_0 = f.subs(x, 0)
continuidade_em_pi = f.subs(x, pi)
continuidade_em_inf = f.limit(x, oo)

print("Continuidade em x=0:", continuidade_em_0)
print("Continuidade em x=pi:", continuidade_em_pi)
print("Continuidade em x=∞:", continuidade_em_inf)
```

```
Continuidade em x=0: 0
Continuidade em x=pi: 0
Continuidade em x=∞: AccumBounds(-1, 1)
```

Neste exemplo, estamos verificando a continuidade da função seno (sin(x)) nos pontos x=0, x=pi e x=infinito. A função subs é usada para substituir os valores de x na função e, em seguida, verificamos se o limite existe em cada ponto.

4. Derivada

4.1. Definição de Derivada

A derivada é um conceito fundamental do cálculo e representa a taxa de variação instantânea de uma função em relação à sua variável independente. Em outras palavras, a derivada mede a inclinação da função em um ponto específico. A derivada de uma função f(x) em relação a x é denotada por f'(x) ou df/dx.

A derivada de uma função f(x) no ponto x é definida como o limite do quociente incremental quando a variável independente (x) se aproxima de zero:

```
f'(x) = lim_(h → 0) [f(x + h) - f(x)] / h
```

Essa expressão representa a taxa de variação instantânea de f(x) em relação a x quando x se aproxima de um valor específico.

Interpretação Geométrica
Geometricamente, a derivada de uma função em um ponto representa a inclinação da reta tangente à curva da função nesse ponto. Ela indica como a função está mudando em torno desse ponto.

Exemplo de Codificação em Jupyter Notebook
Aqui está um exemplo de como calcular a derivada de uma função em Python usando a biblioteca SymPy em um Jupyter Notebook:

```
from sympy import Symbol, diff, exp

# Definindo uma variável simbólica
x = Symbol('x')

# Calculando a derivada
derivada = diff(exp(x**2), x)

print("Derivada:", derivada)
```

```
Derivada: 2*x*exp(x**2)
```

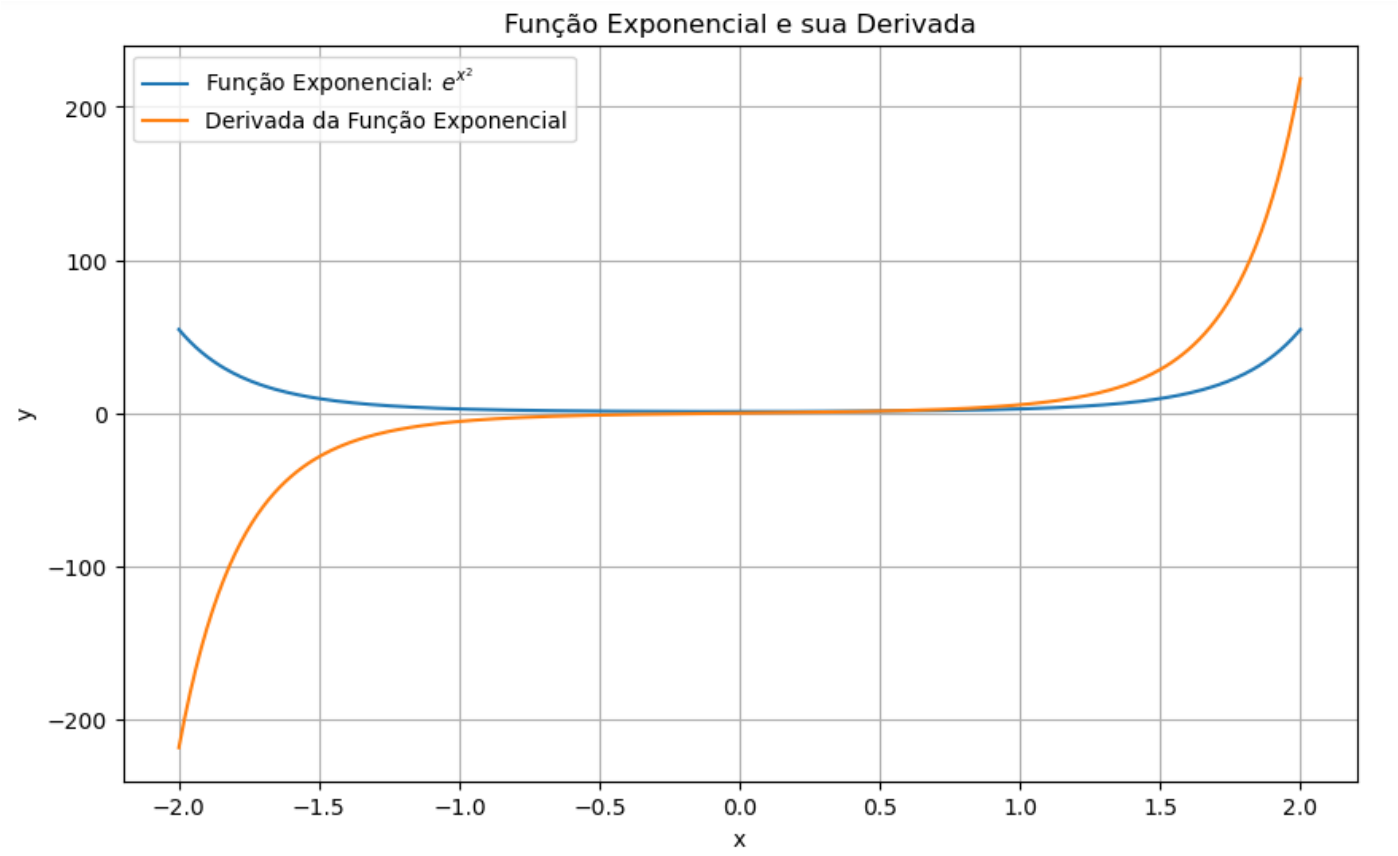
Neste exemplo, estamos calculando a derivada da função exponencial exp(x^2) em relação a x. A biblioteca SymPy é usada para realizar esse cálculo.

Uma vez calculado, podemos ver o comportamento dessa relação entre essa função exponencial e x com o seguinte script:

```
import numpy as np
import matplotlib.pyplot as plt
from sympy import Symbol, diff, exp

# Definindo uma variável simbólica
x = Symbol('x')

# Calculando a derivada da função
derivada = diff(exp(x**2), x)

print("Derivada:", derivada)

# Convertendo a derivada em uma função lambda para avaliação numérica
derivada_func = lambda x_val: derivada.subs(x, x_val)
```

```
# Intervalo de valores de x para o gráfico
x_vals = np.linspace(-2, 2, 400)
y_exp = np.exp(x_vals**2)
y_deriv = np.vectorize(derivada_func)(x_vals)

# Configurando o layout do gráfico
plt.figure(figsize=(10, 6))
plt.title("Função Exponencial e sua Derivada")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)

# Plotando a função exponencial e sua derivada
plt.plot(x_vals, y_exp, label="Função Exponencial: $e^{x^2}$")
plt.plot(x_vals, y_deriv, label="Derivada da Função Exponencial")
plt.legend()

# Mostrando o gráfico
plt.show()
```

O script calculará a derivada da função exp(x**2) em relação a x, criará um gráfico mostrando a função exponencial e sua derivada no intervalo de valores de x especificado e exibirá o gráfico resultante.



Lembre-se de que a função sympy.diff retorna a derivada simbólica da função, que é representada como uma expressão simbólica. A conversão para uma função numérica é realizada usando lambda e subs para avaliar a derivada em pontos específicos.

Observando o gráfico, temos: A função exponencial exp(x**2) cresce muito rapidamente à medida que x aumenta. Isso ocorre porque o valor de x**2 cresce mais rápido do que x, e quando você aplica a função exponencial, os valores se tornam muito grandes muito rapidamente. Portanto, a curva da função exponencial no gráfico será uma curva ascendente acentuada.

A derivada da função exponencial exp(x**2) em relação a x é calculada e representada no gráfico como a linha que representa a derivada. A derivada dessa função é uma expressão bastante complexa, mas ela também cresce rapidamente à medida que x aumenta. Portanto, a curva da derivada será uma curva ascendente, assim como a função exponencial original.

O ponto em que a derivada atinge o valor zero (ponto onde a curva da derivada cruza o eixo x) é um ponto crítico da função exponencial. Isso ocorre porque a derivada nos diz onde a função tem uma taxa de variação zero, ou seja, onde ela muda de concavidade. No caso da função exponencial exp(x**2), esse ponto crítico está próximo a x = 0.

Lembre-se de que a derivada da função exponencial é a própria função exponencial multiplicada pelo valor da variável, ou seja, a derivada de exp(x**2) em relação a x é 2*x*exp(x**2). Isso contribui para o crescimento rápido da derivada à medida que x aumenta.

Em resumo, o gráfico ilustra visualmente como a função exponencial exp(x**2) cresce rapidamente à medida que x aumenta e como a sua derivada também cresce rapidamente, com um ponto crítico próximo a x = 0. Isso demonstra o comportamento característico de uma função exponencial e sua derivada.

4.2. Regras de Derivação

As regras de derivação são ferramentas poderosas para simplificar o cálculo de derivadas de funções mais complexas. Essas regras são amplamente utilizadas na Ciência de Dados para calcular derivadas de forma mais eficiente.

Regras Básicas de Derivação
Existem algumas regras básicas de derivação que podem ser aplicadas em várias situações. Alguns exemplos incluem:

Regra da soma e da diferença:
```
(d/dx) [f(x) ± g(x)] = f'(x) ± g'(x)
```

Regra do produto:
```
(d/dx) [f(x) * g(x)] = f'(x) * g(x) + f(x) * g'(x)
```

Regra do quociente:
```
(d/dx) [f(x) / g(x)] = [f'(x) * g(x) - f(x) * g'(x)] / [g(x)]^2
```

Regra da cadeia:
```
(d/dx) [f(g(x))] = f'(g(x)) * g'(x)
```

Exemplo de Codificação em Jupyter Notebook

Aqui está um exemplo de como aplicar as regras de derivação em Python usando a biblioteca SymPy em um Jupyter Notebook:

```
from sympy import Symbol, diff, sin, cos

# Definindo uma variável simbólica
x = Symbol('x')

# Aplicando as regras de derivação
f = sin(x) * cos(x)
derivada = diff(f, x)

print("Derivada:", derivada)
```

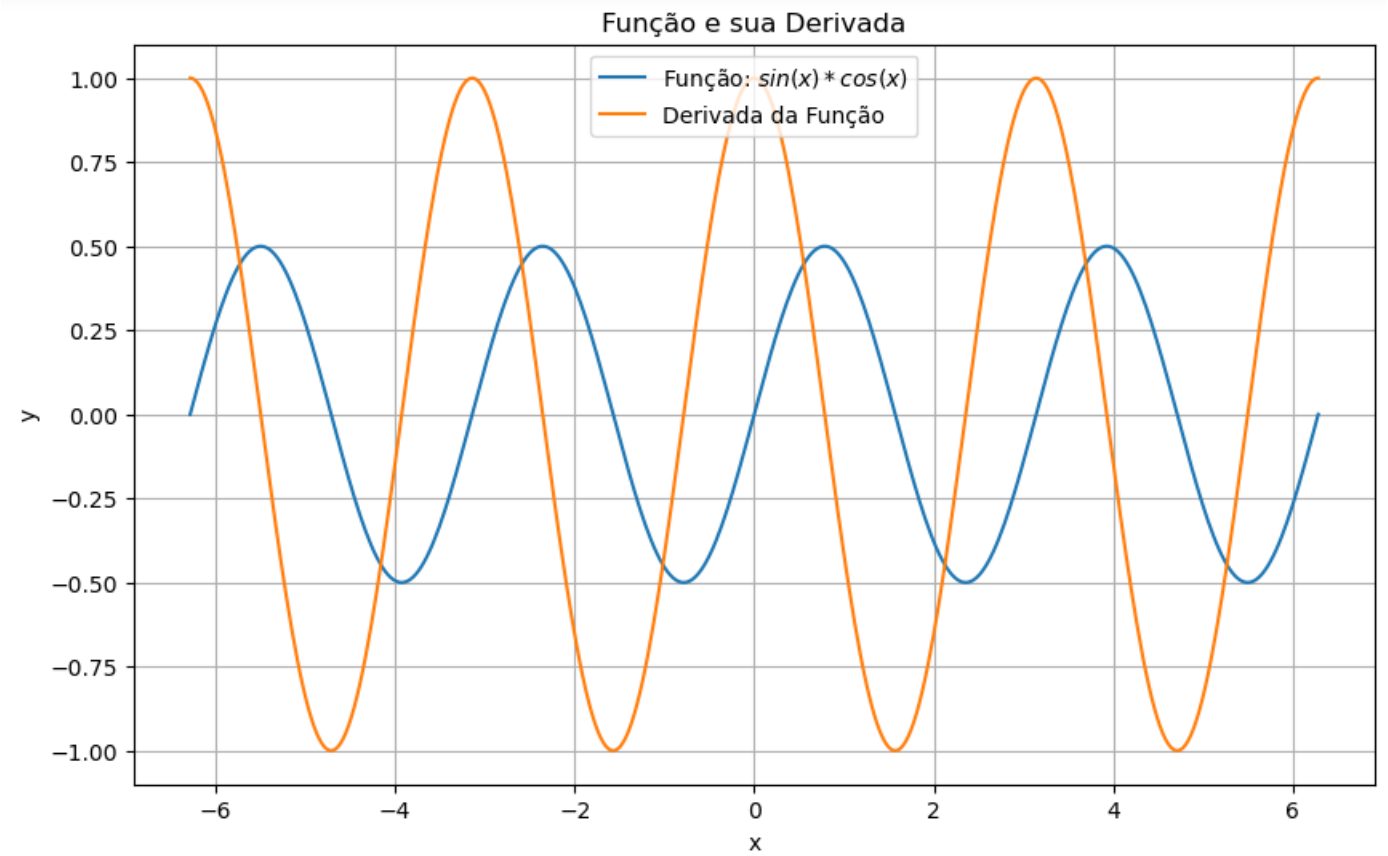
```
Derivada: -sin(x)**2 + cos(x)**2
```

Neste exemplo, estamos calculando a derivada da função sin(x) * cos(x) em relação a x. A biblioteca SymPy é usada para simplificar a expressão e aplicar as regras de derivação.

O código a seguir ilustra como é o comportamento dessa derivada graficamente:

```
import numpy as np
import matplotlib.pyplot as plt
from sympy import Symbol, diff, sin, cos

# Definindo uma variável simbólica
x = Symbol('x')

# Definindo a função
f = sin(x) * cos(x)

# Calculando a derivada
derivada = diff(f, x)

print("Derivada:", derivada)

# Convertendo a derivada em uma função lambda para avaliação numérica
derivada_func = lambda x_val: derivada.subs(x, x_val)

# Intervalo de valores de x para o gráfico
x_vals = np.linspace(-2 * np.pi, 2 * np.pi, 400)
y_function = np.sin(x_vals) * np.cos(x_vals)
y_derivative = np.vectorize(derivada_func)(x_vals)

# Configurando o layout do gráfico
plt.figure(figsize=(10, 6))
plt.title("Função e sua Derivada")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)

# Plotando a função e sua derivada
plt.plot(x_vals, y_function, label="Função: $sin(x) * cos(x)$")
plt.plot(x_vals, y_derivative, label="Derivada da Função")
plt.legend()

# Mostrando o gráfico
plt.show()
```

O script calculará a derivada da função sin(x) * cos(x), criará um gráfico mostrando a função e sua derivada no intervalo de valores de x especificado e exibirá o gráfico resultante.



A função sin(x) * cos(x) possui comportamento oscilatório, e sua derivada mostrará a taxa de variação dessas oscilações ao longo do intervalo. A derivada será zero nos pontos onde a função cruza o eixo x (máximos e mínimos locais) e atingirá valores máximos nos pontos médios entre esses extremos. O gráfico ilustrará esse comportamento.

4.3. Derivadas de Funções Elementares

As funções elementares são funções básicas que são amplamente utilizadas na Ciência de Dados. É importante conhecer as derivadas dessas funções para poder calcular derivadas de funções mais complexas.

Derivadas de Funções Elementares Comuns
Aqui estão algumas das derivadas das funções elementares mais comuns:

Derivada da função constante:
```
(d/dx) [c] = 0
```

Derivada da função linear:
```
(d/dx) [ax + b] = a
```

Derivada da função exponencial:
```
(d/dx) [e^x] = e^x
```

Derivada da função logarítmica:
```
(d/dx) [ln(x)] = 1/x
```

Derivada da função seno:
```
(d/dx) [sin(x)] = cos(x)
```

Derivada da função cosseno:
```
(d/dx) [cos(x)] = -sin(x)
```

Exemplo de Codificação em Jupyter Notebook

Aqui está um exemplo de como calcular as derivadas de funções elementares em Python usando a biblioteca SymPy em um Jupyter Notebook:

```
from sympy import Symbol, diff, exp, log, sin, cos

# Definindo uma variável simbólica
x = Symbol('x')

# Calculando as derivadas de funções elementares
derivada_constante = diff(5, x)
derivada_linear = diff(3*x + 2, x)
derivada_exponencial = diff(exp(x), x)
derivada_logaritmica = diff(log(x), x)
derivada_seno = diff(sin(x), x)
derivada_cosseno = diff(cos(x), x)

print("Derivada da função constante:", derivada_constante)
print("Derivada da função linear:", derivada_linear)
print("Derivada da função exponencial:", derivada_exponencial)
print("Derivada da função logarítmica:", derivada_logaritmica)
print("Derivada da função seno:", derivada_seno)
print("Derivada da função cosseno:", derivada_cosseno)
```

```
Derivada da função constante: 0
Derivada da função linear: 3
Derivada da função exponencial: exp(x)
Derivada da função logarítmica: 1/x
Derivada da função seno: cos(x)
Derivada da função cosseno: -sin(x)
```

Neste exemplo, estamos calculando as derivadas das funções elementares e suas respectivas variáveis simbólicas. A biblioteca SymPy é usada para simplificar as expressões e calcular as derivadas.

5. Aplicações de Derivadas

A derivada tem várias aplicações importantes em diversas áreas da ciência, matemática e engenharia. Algumas das principais aplicações incluem:

- **Determinação de Taxas de Variação e Velocidades Instantâneas**: A derivada é usada para calcular taxas de variação de uma quantidade em relação a outra, como velocidades instantâneas em física e taxas de crescimento em economia.

- **Encontrar Pontos Críticos e Extremos**: Os pontos críticos são pontos onde a derivada é igual a zero ou não existe, e podem corresponder a mínimos locais, máximos locais ou pontos de inflexão em uma função.

- **Análise de Concavidade e Pontos de Inflexão**: A segunda derivada (derivada da derivada) é usada para analisar a concavidade de uma função e identificar pontos de inflexão.

- **Resolução de Problemas de Otimização**: A derivada é usada para encontrar valores ótimos em problemas de maximização e minimização.

5.1. Taxa de Variação Instantânea e Velocidades Instantâneas

Vamos considerar a seguinte função que descreve o movimento de um objeto em uma trajetória retilínea:

```
s(t) = 5t^2 - 2t + 10
```

Vamos calcular a derivada dessa função em relação ao tempo (t) e analisar suas aplicações.

```
import sympy as sp
import matplotlib.pyplot as plt
import numpy as np

# Definindo a variável t
t = sp.symbols('t')

# Definindo a função s(t)
s_t = 5 * t**2 - 2 * t + 10

# Calculando a derivada da função em relação ao tempo (t)
derivada_s_t = sp.diff(s_t, t)

# Exibindo a derivada
print("Derivada de s(t) em relação a t:")
print(derivada_s_t)

# Plotando o gráfico da função s(t) e sua derivada
t_values = np.linspace(-2, 3, 100)
s_values = [s_t.subs(t, value) for value in t_values]
derivada_values = [derivada_s_t.subs(t, value) for value in t_values]

plt.plot(t_values, s_values, label='s(t) = 5t^2 - 2t + 10')
plt.plot(t_values, derivada_values, label="s'(t) = 10t - 2", linestyle='dashed')
plt.xlabel('t')
plt.ylabel('s(t), s\'(t)')
plt.title('Gráfico de s(t) e sua Derivada')
plt.legend()
plt.grid(True)
plt.show()
```

Neste exemplo, calculamos a derivada da função s(t) = 5t^2 - 2t + 10 em relação ao tempo (t) usando a biblioteca sympy. Em seguida, plotamos o gráfico da função original e sua derivada.

Observe que a derivada s'(t) = 10t - 2 representa a taxa de variação instantânea da posição do objeto em relação ao tempo. A partir do gráfico, podemos analisar a inclinação da função em diferentes pontos e entender o comportamento do movimento do objeto.

5.2 Encontrar Pontos Críticos e Extremos (Ex: Máximos e Mínimos Locais)

Uma aplicação comum das derivadas é a identificação de máximos e mínimos locais de uma função. Para encontrar esses pontos, podemos procurar os pontos críticos da função, onde a derivada se iguala a zero ou não existe.

Máximo local: ocorre quando a derivada muda de positiva para negativa.
Mínimo local: ocorre quando a derivada muda de negativa para positiva.

```
from sympy import Symbol, diff, solve, sin

# Definindo uma variável simbólica
x = Symbol('x')

# Exemplo 1: Encontrando máximos e mínimos locais
f = x**3 - 6*x**2 + 9*x + 2
derivada = diff(f, x)
pontos_criticos = solve(derivada, x)

print("Derivada:", derivada)
print("Pontos críticos:", pontos_criticos)
```

5.3. Pontos de Inflexão

Os pontos de inflexão são pontos em uma curva onde a concavidade muda. Esses pontos podem ser identificados encontrando as soluções da segunda derivada da função, onde a concavidade muda de côncava para convexa ou vice-versa.

```
from sympy import Symbol, diff, solve, sin

# Definindo uma variável simbólica
x = Symbol('x')

# Exemplo 2: Encontrando pontos de inflexão
g = sin(x)
segunda_derivada = diff(g, x, 2)
pontos_inflexao = solve(segunda_derivada, x)

print("Segunda derivada:", segunda_derivada)
print("Pontos de inflexão:", pontos_inflexao)
```

5.4. Otimização de Funções
A otimização de funções envolve encontrar o valor máximo ou mínimo de uma função em um determinado intervalo. Isso é particularmente útil na Ciência de Dados para problemas de modelagem e tomada de decisões. Podemos usar as derivadas para identificar os pontos críticos da função e, em seguida, avaliar se esses pontos são máximos ou mínimos locais.

```
from sympy import Symbol, diff, solve, sin

# Definindo uma variável simbólica
```

```
x = Symbol('x')

# Exemplo 4: Otimização de função
i = x**2 - 4*x + 3
derivada_otimizacao = diff(i, x)
pontos_criticos_otimizacao = solve(derivada_otimizacao, x)

print("Derivada para otimização:", derivada_otimizacao)
print("Pontos críticos para otimização:", pontos_criticos_otimizacao)
```

Nestes exemplos, estamos aplicando as aplicações de derivadas em quatro casos diferentes. Calculamos a derivada, encontramos pontos críticos, pontos de inflexão, taxa de variação instantânea e pontos críticos para otimização de funções.